DATABASE MANAGEMENT SYSTEMS

CT 652 | BE / BCT | Year III / Part II

Tribhuvan University — Institute of Engineering

CHAPTER 4: DATABASE CONSTRAINTS & NORMALIZATION

Question Bank with Comprehensive Answers — FDs, Armstrong's Axioms, 1NF/2NF/3NF/BCNF, 4NF, Lossless Decomposition

SYLLABUS OVERVIEW — Unit 4

Unit	Topic	Hours	Marks
1	Introduction	3 hrs	4
2	Data Models	7 hrs	12
3	Relational Languages and Relational Model	7 hrs	12
4	Database Constraints and Normalization	6 hrs	12
5	Query Processing and Optimization	4 hrs	8
6	File Structure and Hashing	4 hrs	8
7	Transactions Processing and Concurrency Control	6 hrs	12
8	Crash Recovery	4 hrs	6
9	Advanced Database Concepts	4 hrs	6
Total		**45 hrs**	**80**

Chapter 4 Topics (6 Hours | 12 Marks)

- 4.1 Functional Dependencies
- 4.2 Armstrong's Axioms
- 4.3 Normal Forms: 1NF, 2NF, 3NF, BCNF
- 4.4 Lossless Join Decomposition
- 4.5 Dependency Preservation
- 4.6 4NF, 5NF — Multi-valued and Join Dependencies

CHAPTER 4 — Questions & Comprehensive Answers

Q: Define functional dependency. Explain trivial, non-trivial, full, partial, and transitive functional dependencies with examples.
Asked: 2078 Poush; 2077 Chaitra; 2075 Baisakh; 2074; 2072 Magh; 2070–2081 (all years) | Marks: [4]

Answer

Functional Dependency (FD)

A functional dependency (FD) is a constraint between two sets of attributes in a relation. Given a relation schema R, a functional dependency X → Y (read: 'X functionally determines Y') holds if and only if: for any two tuples t1 and t2 in any valid instance of R, whenever t1[X] = t2[X] (they agree on X), it must follow that t1[Y] = t2[Y] (they also agree on Y).

In other words, knowing the value of X uniquely determines the value of Y. X is called the determinant and Y is the dependent.

Notation and Definition

```
X → Y  means 'X functionally determines Y'
        equivalently: 'Y is functionally dependent on X'

Formally: ∀ t1, t2 ∈ r(R):  t1[X] = t2[X]  □  t1[Y] = t2[Y]
```

Types of Functional Dependencies

1. Trivial FD

An FD X → Y is trivial if Y ⊆ X (Y is a subset of X). It holds in every relation by definition and conveys no useful information.

```
Examples of trivial FDs:
  AB → A          (A is a subset of AB)
  AB → B          (B is a subset of AB)
  ABC → BC        (BC ⊆ ABC)
  A  → A          (always true)
```

2. Non-Trivial FD

An FD X → Y is non-trivial if Y □ X (at least one attribute in Y is not in X). These are the meaningful FDs in schema design.

```
Examples of non-trivial FDs:
  empid → name          (name is NOT a subset of {empid})
  student_id → address
  AB → C                (C □ {A,B})
```

3. Full Functional Dependency

An FD X → Y is a full functional dependency if the removal of any single attribute from X makes the FD no longer hold. That is, no proper subset of X can determine Y.

```
Example: (student_id, course_id) → grade
  student_id alone does NOT determine grade
  course_id alone does NOT determine grade
  Only together do they determine grade → FULL FD

Example: (empid, project_id) → hours_worked   [FULL FD]
  Neither empid nor project_id alone determines hours_worked
```

4. Partial Functional Dependency

An FD X → Y is a partial functional dependency if some proper subset of X can also determine Y. Partial dependencies always involve a composite key — a non-key attribute is determined by only part of the key.

```
Relation R(student_id, course_id, student_name, grade)
  PK = (student_id, course_id)
  FD: student_id → student_name  [PARTIAL FD]
       student_name depends only on PART of the composite key
  FD: (student_id, course_id) → grade  [FULL FD]

Partial FDs cause 2NF violations — they must be removed in 2NF.
```

5. Transitive Functional Dependency

An FD X → Z is a transitive functional dependency if there exists an attribute set Y such that X → Y and Y → Z, and Y is not a superkey, and Y does not determine X (i.e., Y □ X). The dependency of Z on X is 'indirect' — through Y.

```
Example:
  empid → dept_id        (employee determines department)
  dept_id → dept_name    (department determines dept name)
  ∴ empid → dept_name    [TRANSITIVE FD]
   dept_name transitively depends on empid through dept_id

Transitive FDs cause 3NF violations — they must be removed in 3NF.
```

FD Type	Condition	Normal Form Violation
Trivial	Y □ X (Y is subset of determinant)	None — always valid
Non-trivial	Y □ X (useful constraint)	Basis of all normalization
Full FD	No proper subset of X determines Y	Satisfied in 2NF
Partial FD	A proper subset of composite key determines non-key attr	Violates 2NF
Transitive FD	X→Y→Z where Y is not a superkey and Y□X	Violates 3NF

Q: What are Armstrong's Axioms? State and explain each with an example. Prove the derived rules (union, decomposition, pseudotransitivity). Show that A→B, A→C implies A→BC.

Asked: 2081 Chaitra; 2080 Ashwin; 2079 Ashwin; 2076; 2075; 2073 Magh; 2071; 2070 (all years) | Marks: [4]

Answer

Armstrong's Axioms

Armstrong's Axioms (proposed by William Armstrong, 1974) are a complete and sound set of inference rules for deriving all functional dependencies implied by a given set F of FDs. They are used to compute the closure of FD sets.

SOUNDNESS: Every FD derivable using these axioms actually holds in every relation satisfying F.
COMPLETENESS: Every FD that holds in all relations satisfying F can be derived using these axioms.
These two properties together mean Armstrong's Axioms are exactly sufficient — no more, no less.

Three Primary (Primitive) Axioms

Axiom 1: Reflexivity (Trivial Rule)

```
If Y ⊆ X, then X → Y

Example: {A, B, C} → {A, B}     because {A,B} ⊆ {A,B,C}
         AB → A                  (trivially true)
         empid, name → empid     (trivially true)
```

Axiom 2: Augmentation

```
If X → Y, then XZ → YZ  (for any attribute set Z)

Example: If empid → name, then (empid, dept) → (name, dept)
         If A → B,       then  AC → BC

Intuition: Adding the same attributes to both sides preserves the FD.
```

Axiom 3: Transitivity

```
If X → Y and Y → Z, then X → Z

Example: empid → dept_id  and  dept_id → dept_name
         ∴ empid → dept_name

         A → B  and  B → C  □  A → C
```

Derived Rules (from the 3 primary axioms)

Rule 4: Union (Additivity)

```
If X → Y and X → Z, then X → YZ

Proof:
  (1) X → Y          [given]
  (2) X → Z          [given]
  (3) X → XY         [augment (1) with X: XX→XY, but XX=X so X→XY]
       More precisely: by augmentation on (1): XX → YX, i.e., X → XY
  (4) XY → YZ        [augment (2) with Y: XY → YZ]
  (5) X → YZ         [transitivity on (3) and (4)]

Example: A→B and A→C  □  A→BC
  Proof: A→B (given); A→C (given)
         A→AB (augment A→B with A)
         AB→BC (augment A→C with B)
         A→BC (transitivity: A→AB and AB→BC)
```

Rule 5: Decomposition (Splitting)

```
If X → YZ, then X → Y and X → Z

Proof:
  (1) X → YZ          [given]
  (2) YZ → Y          [reflexivity: Y ⊆ YZ]
  (3) YZ → Z          [reflexivity: Z ⊆ YZ]
  (4) X → Y           [transitivity on (1) and (2)]
  (5) X → Z           [transitivity on (1) and (3)]

Example: A→BC  □  A→B  and  A→C
```

Rule 6: Pseudo-Transitivity

```
If X → Y and WY → Z, then WX → Z

Proof:
  (1) X → Y           [given]
  (2) WY → Z          [given]
  (3) WX → WY         [augment (1) with W: WX→WY]
  (4) WX → Z          [transitivity on (3) and (2)]
```

Q: Define the closure of a set of attributes. Find the closure of attribute set {A} and complete closure F+ given FDs: F = {A→B, B→C, A→D, D→E}. Also explain canonical cover.

Asked: 2081 Ashwin; 2075 Bhadra; 2074; 2073 Magh; 2072 Ashwin; 2071 Magh | Marks: [2+2+4]

Answer

Closure of an Attribute Set

Given a set of FDs F and an attribute set X, the closure of X under F (written X+) is the set of all attributes that can be functionally determined from X using the FDs in F (and Armstrong's Axioms).

```
X+ = { A | X → A can be derived from F using Armstrong's Axioms }
```

Algorithm to Compute X+

```
Input: Attribute set X, FD set F
Output: X+ (closure of X)

1. Initialize:  X+ = X   (start with X itself — reflexivity)
2. Repeat until no change:
   For each FD (Y → Z) in F:
       If Y □ X+:         (if all attributes of Y are already in X+)
           X+ = X+ ∪ Z    (add Z to the closure)
3. Return X+
```

Worked Example: F = {A→B, B→C, A→D, D→E}

Find closure of {A}:

```
Initial:  {A}+ = {A}

Iteration 1:
  A → B:   A ⊆ {A}?  YES  →  add B:    {A}+ = {A, B}
  B → C:   B ⊆ {A}?  NO   →  skip
  A → D:   A ⊆ {A}?  YES  →  add D:    {A}+ = {A, B, D}
  D → E:   D ⊆ {A}?  NO   →  skip

Iteration 2:
  A → B:   A ⊆ {A,B,D}?  YES  →  B already in, no change
  B → C:   B ⊆ {A,B,D}?  YES  →  add C:    {A}+ = {A, B, C, D}
  A → D:   A ⊆ {A,B,D}?  YES  →  D already in, no change
  D → E:   D ⊆ {A,B,D}?  YES  →  add E:    {A}+ = {A, B, C, D, E}

Iteration 3: No new attributes added. Stop.

Result:  {A}+ = {A, B, C, D, E}
  Since {A}+ includes ALL attributes, A is a candidate key (superkey).
```

Find closures of other attributes:

```
{B}+: B→C  →  {B,C}     No more applicable FDs.   {B}+ = {B, C}
{D}+: D→E  →  {D,E}     No more applicable FDs.   {D}+ = {D, E}
{C}+: No FD with C alone on LHS.                   {C}+ = {C}
{E}+: No FD with E alone on LHS.                   {E}+ = {E}
```

Closure of FD Set (F+)

The closure of a set of FDs F (written F+) is the set of ALL functional dependencies that can be derived from F using Armstrong's Axioms. For a relation with n attributes, F+ can have up to 2^n × 2^n FDs (very large). We rarely compute F+ explicitly — instead we use attribute closure.

Canonical Cover (Fc)

A canonical cover Fc of F is a minimal set of FDs equivalent to F (i.e., Fc+ = F+) with these properties:

- No FD in Fc has an extraneous attribute on the left or right side
- Every left-hand side of an FD in Fc is unique
- Fc is minimal — removing any FD from Fc changes F+

Algorithm to Find Canonical Cover:

```
1. Apply decomposition: break each X → YZ into X → Y and X → Z
2. Remove extraneous attributes from LHS of each FD:
   Attribute A in X → Y is extraneous if (X - {A}) → Y can still be derived from F
3. Remove extraneous attributes from RHS of each FD:
   Attribute B in X → Y is extraneous if X → Y - {B} can be derived from (F -
{X→Y}) ∪ {X→Y-{B}}
4. Remove any redundant FD X → Y where Y can be derived from F - {X→Y}
5. Combine FDs with same LHS: if A → B and A → C, combine to A → BC
```

Q: What is normalization? What anomalies does redundancy cause? Explain insertion, deletion, and update anomalies with examples.

Asked: 2078 Baishakh; 2073 Bhadra; 2078 Chaitra; 2070 Bhadra | Marks: [4]

Answer

Normalization

Normalization is the process of organizing a relational database schema to reduce data redundancy and eliminate undesirable data anomalies. It involves decomposing a relation into smaller, well-structured relations that satisfy specific normal form conditions, while preserving:

- All original data (lossless decomposition)
- All original constraints (dependency preservation)

Problems Caused by Redundancy

Consider a poorly designed relation (universal relation — everything in one table):

```
EmpProject(ENo, EName, DNo, DName, MgrNo, PNo, PName, HrsWorked)

Sample data:
ENo  EName   DNo  DName    MgrNo  PNo  PName     HrsWorked
E01  Alice   D01  Sales    M01    P01  Alpha      10
E01  Alice   D01  Sales    M01    P02  Beta       20
E02  Bob     D01  Sales    M01    P01  Alpha      15
E03  Carol   D02  Research M02    P03  Gamma       8
```

1. Insertion Anomaly

It is impossible to insert some data without inserting other, unrelated data.

- Problem: We cannot add a new department (DNo, DName, MgrNo) to the database unless that department already has at least one employee assigned to a project.
- Example: Cannot insert D03 (IT, M03) without an employee and project.
- Root cause: Department info is stored redundantly with employee-project data.

2. Deletion Anomaly

Deleting one piece of information unintentionally destroys other unrelated information.

- Problem: If employee E03 is the only employee in the Research department (D02) and we delete E03's record, we also lose all information about department D02.
- Example: DELETE WHERE ENo = 'E03' → D02 (Research, M02) is lost forever.
- Root cause: Department data is co-located with employee-project data.

3. Update Anomaly

Updating one logical piece of data requires updating multiple rows, causing inconsistency if some rows are missed.

- Problem: If the manager of D01 changes from M01 to M04, we must update EVERY row where DNo = 'D01'. If any row is missed, the database becomes inconsistent.
- Example: Alice appears in two rows — if only one is updated, different rows show different managers for the same department.
- Root cause: The same fact (D01's manager) is stored multiple times.

Solution — Normalization

Normalization removes these anomalies by eliminating partial and transitive FDs. The result is a set of well-structured relations (normal forms) that store each fact in exactly one place. The trade-off is that queries may require joining more tables — but data integrity is guaranteed.

Q: Define 1NF, 2NF, and 3NF. Explain each with examples. What does each normal form eliminate?

Asked: 2081 Chaitra; 2080 Ashwin; 2079 Chaitra; 2078 Poush; 2078 Baishakh; 2073 Bhadra; 2070 Bhadra — all years | Marks: [6]

Answer

First Normal Form (1NF)

A relation R is in 1NF if and only if every attribute of R contains only atomic (indivisible, single-valued) values, and there are no repeating groups or arrays. The domain of each attribute must contain only atomic values.

1NF Violations:

- Multi-valued attribute: Phone = {9841000001, 9851000002}
- Composite attribute: Address = (Street, City, Zip)
- Repeating groups: Course1, Course2, Course3 as separate columns

Example — Not in 1NF:

```
Student(student_id, name, courses)

S01  Alice   {Math, Physics, CS}    ← multi-valued: violates 1NF
S02  Bob     {Math, English}
```

Convert to 1NF:

```
Student_Course(student_id, name, course)    ← one row per (student, course)
S01  Alice  Math
S01  Alice  Physics
S01  Alice  CS
S02  Bob    Math
S02  Bob    English
```

Second Normal Form (2NF)

A relation R is in 2NF if it is in 1NF AND every non-prime attribute (attribute not part of any candidate key) is fully functionally dependent on every candidate key. There must be NO partial functional dependencies.

> *Note: 2NF only matters when the relation has a composite candidate key. If the PK is a single attribute, a relation in 1NF is automatically in 2NF.*

Example — In 1NF but NOT in 2NF:

```
Orders(order_id, product_id, order_date, product_name, quantity)
  PK = (order_id, product_id)

FDs:
  (order_id, product_id) → order_date, product_name, quantity  [composite key]
  product_id → product_name    ← PARTIAL FD! (part of PK → non-key attr)
  order_id   → order_date      ← PARTIAL FD!
```

Convert to 2NF (eliminate partial FDs):

```
Order_Header(order_id, order_date)           ← remove order_date partial FD
Product(product_id, product_name)            ← remove product_name partial FD
Order_Line(order_id, product_id, quantity)   ← only full FDs remain

Now (order_id,product_id)→quantity is the only FD in Order_Line — fully
functional.
```

Third Normal Form (3NF)

A relation R is in 3NF if it is in 2NF AND there are no transitive functional dependencies. An FD X → A (where A is a non-prime attribute) violates 3NF if X is not a superkey of R.

More formally, for every non-trivial FD X → Y in R, at least one of the following must hold:

- X is a superkey of R, OR
- Every attribute in Y is a prime attribute (part of some candidate key)

Example — In 2NF but NOT in 3NF:

```
Employee(empid, name, dept_id, dept_name)
  PK = empid

FDs:
  empid → name, dept_id, dept_name    [OK — empid is PK]
  dept_id → dept_name                ← TRANSITIVE FD!
```

```
                                        (empid→dept_id→dept_name)
                                        dept_id is not a superkey
```

Convert to 3NF (eliminate transitive FDs):

```
Employee(empid, name, dept_id)          ← remove dept_name
Department(dept_id, dept_name)          ← new relation for transitive FD

Now: empid→name,dept_id (direct), dept_id→dept_name (direct).
No transitive dependency remains.
```

Normal Form	Condition	Eliminates
1NF	All attributes have atomic values; no repeating groups	Multi-valued and composite attributes
2NF	In 1NF + no partial FDs (non-key attr fully depends on whole key)	Partial functional dependencies
3NF	In 2NF + no transitive FDs (non-key attr depends only on key)	Transitive functional dependencies
BCNF	In 3NF + every determinant is a superkey	All anomalies from FDs; stricter than 3NF

Q: Define Boyce-Codd Normal Form (BCNF). How does it differ from 3NF? Give an example of a relation in 3NF but not in BCNF. Explain the BCNF decomposition algorithm.

Asked: 2079 Chaitra; 2078 Poush; 2078 Baishakh; 2077; 2076; 2072 Magh; 2071; 2070 (all years) | Marks: [4]

Answer

Boyce-Codd Normal Form (BCNF)

A relation R is in BCNF if and only if for every non-trivial functional dependency X → Y in R, X is a superkey of R. BCNF is a stricter version of 3NF — it eliminates all redundancy due to functional dependencies.

```
BCNF condition: For every non-trivial FD X → Y in R:
                X must be a superkey of R

This is stronger than 3NF, which allows X→Y if Y is a prime attribute.
BCNF removes that exception.
```

3NF vs BCNF

Aspect	3NF	BCNF
Condition for FD X→Y (non-trivial)	X is a superkey OR Y is a prime attribute	X must be a superkey (no exception for prime attributes)

Strictness	Less strict — allows some anomalies	Stricter — eliminates all FD-based anomalies
Dependency preservation	Always achievable	May not always preserve all dependencies
Lossless decomposition	Achievable	Always achievable
When they differ	When a prime attribute has a non-trivial FD from a non-superkey	Same relation can violate BCNF but satisfy 3NF

Classic Example: 3NF but NOT BCNF

```
Teaches(student, course, instructor)

Constraints:
  Each instructor teaches exactly one course
  Each student-course pair is taught by exactly one instructor

FDs:
  (student, course) → instructor    [composite CK1]
  (student, instructor) → course    [composite CK2]
  instructor → course               ← violates BCNF!
                                       instructor is NOT a superkey
                                       (instructor alone doesn't identify full
tuple)

Candidate keys: {student,course} and {student,instructor}
All attributes are PRIME (part of some CK).
  → 3NF satisfied (Y=course is a prime attribute)
  → BCNF violated (instructor is not a superkey)
```

BCNF Decomposition Algorithm

```
Input:  Relation schema R, set of FDs F
Output: BCNF decomposition of R

result = {R}
While some schema Ri in result is NOT in BCNF:
    Find a non-trivial FD X → Y in Ri such that X is NOT a superkey of Ri
    Decompose Ri into:
        R1 = X ∪ Y          (contains X and all attributes in Y)
        R2 = Ri - Y         (Ri minus the attributes of Y, keeping X)
    Replace Ri in result with {R1, R2}
    Compute FDs for R1 and R2 by projecting F onto their attributes
Return result
```

Key property: BCNF decomposition is always lossless (information is preserved).
Warning: BCNF decomposition may NOT always preserve all dependencies.
If dependency preservation is critical, use 3NF decomposition instead.

NORMALIZATION WORKED EXAMPLES

Q: Normalize to highest possible normal form: R(student_id, course_id, student_name, instructor_id, instructor_name, grade). FDs: student_id→student_name; course_id→instructor_id; instructor_id→instructor_name; (student_id,course_id)→grade

Asked: 2081 Ashwin (Back Exam) | Marks: [6]

Answer

Given FDs:

```
F1: student_id  → student_name
F2: course_id   → instructor_id
F3: instructor_id → instructor_name
F4: (student_id, course_id) → grade
```

Step 1: Identify Primary Key

```
What determines ALL attributes?
(student_id, course_id)+:
  student_id → student_name            [F1]
  course_id  → instructor_id           [F2]
  instructor_id → instructor_name      [F3]
  (student_id,course_id) → grade       [F4]
  → {student_id,course_id}+ = ALL attributes

PK = (student_id, course_id)
```

Step 2: Check 1NF

All attributes are atomic. Relation is in 1NF. ✓

Step 3: Check 2NF — remove partial FDs

```
PK = (student_id, course_id) [composite]

Partial FDs (non-key attribute depending on part of PK):
  student_id → student_name    ← PARTIAL (depends on student_id only)
  course_id  → instructor_id   ← PARTIAL (depends on course_id only)
  instructor_id → instructor_name  ← not a partial FD (instructor_id is not
part of PK)

Full FD:
  (student_id, course_id) → grade   ✓ FULL

Decompose to eliminate partial FDs:
  R1(student_id, student_name)           F: student_id → student_name
  R2(course_id, instructor_id)           F: course_id → instructor_id
```

```
  R3(student_id, course_id, grade)       F: (sid,cid) → grade
  [instructor_id → instructor_name is carried along in R2]
```

Step 4: Check 3NF — remove transitive FDs

```
Check R1(student_id, student_name):
  PK = student_id; only FD: student_id→student_name. No transitive FD. IN 3NF
✓

Check R2(course_id, instructor_id):
  PK = course_id; FD: course_id→instructor_id. No transitive FD... BUT
  We also have instructor_id→instructor_name — instructor_name not here yet.
  R2 needs instructor_name too? Let's check:
  If R2 = (course_id, instructor_id, instructor_name):
    course_id → instructor_id → instructor_name  ← TRANSITIVE!
  Decompose R2 further:
    R2a(course_id, instructor_id)       F: course_id → instructor_id
    R2b(instructor_id, instructor_name) F: instructor_id → instructor_name

Check R3(student_id, course_id, grade):
  PK = (student_id, course_id); only FD: (sid,cid)→grade. IN 3NF ✓
```

Final 3NF Decomposition:

```
STUDENT    (student_id PK, student_name)
COURSE     (course_id PK, instructor_id FK→INSTRUCTOR)
INSTRUCTOR (instructor_id PK, instructor_name)
ENROLLMENT (student_id FK, course_id FK, grade)   PK=(student_id, course_id)
```

All four relations are in BCNF (each FD has a superkey as determinant).

Q: Normalize to 3NF: R(P#, P_name, Color, Weight, City, S#, S_name, Status, Qty). FDs: P#→P_name,Color,Weight,City; S#→S_name,Status; (P#,S#)→Qty

Asked: 2080 Ashwin (Back); 2073 Magh (Back) — also: R(S#,SName,Status,City,P#,PName,Color,Weight,Qty) | Marks: [6]

Answer

Given FDs:

```
F1: P# → P_name, Color, Weight, City
F2: S# → S_name, Status
F3: (P#, S#) → Qty
```

Step 1: Identify Primary Key

```
(P#, S#)+:
  P# → P_name, Color, Weight, City    [F1]
```

```
  S# → S_name, Status                    [F2]
  (P#,S#) → Qty                          [F3]
  → (P#, S#)+ = ALL attributes  →  PK = (P#, S#)
```

Step 2: 1NF — all attributes atomic. ✓

Step 3: Check 2NF — remove partial FDs

```
PK = (P#, S#) [composite]

Partial FDs:
  P# → P_name, Color, Weight, City   ← depends on P# only (part of PK)
  S# → S_name, Status                ← depends on S# only (part of PK)

Full FD:
  (P#, S#) → Qty                     ✓ full

Decompose:
  R1(P#, P_name, Color, Weight, City)   F: P#→P_name,Color,Weight,City
  R2(S#, S_name, Status)               F: S#→S_name,Status
  R3(P#, S#, Qty)                      F: (P#,S#)→Qty
```

Step 4: Check 3NF — no transitive FDs in any of R1, R2, R3

```
R1: PK=P#; all FDs: P#→{P_name,Color,Weight,City}. No transitive FDs. 3NF ✓
R2: PK=S#; all FDs: S#→{S_name,Status}. No transitive FDs. 3NF ✓
R3: PK=(P#,S#); only FD: (P#,S#)→Qty. 3NF ✓
```

Final 3NF (and BCNF) Decomposition:

```
PART     (P# PK, P_name, Color, Weight, City)
SUPPLIER (S# PK, S_name, Status)
SHIPMENT (P# FK, S# FK, Qty)    PK = (P#, S#)
```

Q: Normalize to 3NF: R(ENo, EName, DNo, DName, MgrNo, PNo, PName, HrsWorked). FDs: ENo→EName,DNo; DNo→DName,MgrNo; PNo→PName; (ENo,PNo)→HrsWorked

Asked: 2079 Ashwin; 2078 Chaitra; 2075 Baisakh (all similar) | Marks: [6–8]

Answer

Given FDs:

```
F1: ENo → EName, DNo
F2: DNo → DName, MgrNo
F3: PNo → PName
F4: (ENo, PNo) → HrsWorked
```

Step 1: Identify Primary Key

```
(ENo, PNo)+:
  ENo → EName, DNo                  [F1]
  DNo → DName, MgrNo                [F2]
  PNo → PName                       [F3]
  (ENo,PNo) → HrsWorked             [F4]
  → covers ALL attributes  →  PK = (ENo, PNo)
```

Step 2: 1NF ✓ (all atomic)

Step 3: Remove Partial FDs (→ 2NF)

```
PK = (ENo, PNo) [composite]

Partial FDs:
  ENo → EName, DNo              ← depends on ENo only
  DNo → DName, MgrNo            ← DNo is not part of PK; handled as transitive
below
  PNo → PName                   ← depends on PNo only

Full FD:
  (ENo, PNo) → HrsWorked        ✓ full

Decompose (2NF):
  R1(ENo, EName, DNo)           F: ENo→EName,DNo
  R2(PNo, PName)                F: PNo→PName
  R3(ENo, PNo, HrsWorked)       F: (ENo,PNo)→HrsWorked
  [DNo→DName,MgrNo is still in R1 as transitive dependency]
```

Step 4: Remove Transitive FDs (→ 3NF)

```
Check R1(ENo, EName, DNo):
  ENo → DNo → DName, MgrNo   ← TRANSITIVE FD!
  (DName and MgrNo are not in R1 yet, but DNo→DName,MgrNo exists)

  Split R1:
    R1a(ENo, EName, DNo)      F: ENo→EName,DNo  [DNo is FK]
    R1b(DNo, DName, MgrNo)    F: DNo→DName,MgrNo

R2(PNo, PName): PK=PNo; PNo→PName. No transitive FD. 3NF ✓
R3(ENo, PNo, HrsWorked): PK=(ENo,PNo); (ENo,PNo)→HrsWorked. 3NF ✓
```

Final 3NF Decomposition:

```
EMPLOYEE   (ENo PK, EName, DNo FK→DEPARTMENT)
DEPARTMENT (DNo PK, DName, MgrNo)
PROJECT    (PNo PK, PName)
WORKS_ON   (ENo FK, PNo FK, HrsWorked)    PK = (ENo, PNo)
```

Q: Normalize R(A,B,C,D,E) with FDs: A→BC, B→D, A→E to 3NF.
Asked: 2082 Kartik (Back Exam) | Marks: [6]

Answer

Given FDs:

```
F1: A → B, C
F2: B → D
F3: A → E
```

Step 1: Identify PK

```
{A}+:
  A → B, C        [F1]   →  {A,B,C}
  B → D           [F2]   →  {A,B,C,D}
  A → E           [F3]   →  {A,B,C,D,E}
  {A}+ = {A,B,C,D,E} = all attributes  →  PK = A
```

Step 2: 1NF ✓

Step 3: 2NF — PK is single attribute A, so no partial FDs possible. 2NF ✓

Step 4: Check 3NF — remove transitive FDs

```
Transitive FDs (non-key → non-key):
  A → B → D   ← TRANSITIVE (A→B→D; B is not a superkey; B□A)

Non-transitive FDs from A (direct):
  A → B, C, E  (direct from PK)
  B → D        (transitive through B)

Decompose to remove transitive FD B→D:
  R1(A, B, C, E)     F: A→B, A→C, A→E
  R2(B, D)           F: B→D
```

Final 3NF Decomposition:

```
R1(A PK, B, C, E)      F: A→B, A→C, A→E
R2(B PK, D)            F: B→D

Check BCNF:
R1: determinant A is PK (superkey). BCNF ✓
R2: determinant B is PK (superkey). BCNF ✓
→ Decomposition is also in BCNF.
```

Q: Normalize R(A,B,C,D,E) with FDs: A→BC, CD→E, B→D, E→A to 3NF.
Asked: 2076 Baisakh (Back Exam) | Marks: [4]

Answer

Find all candidate keys:

```
{A}+:  A→B,C [F1]; B→D [F3]; CD→E [F2; C,D in closure]: {A,B,C,D,E} = ALL  → A
is a CK
{E}+:  E→A [F4]; A→B,C [F1]; B→D [F3]; CD→E [F2]: {A,B,C,D,E} = ALL  → E is a
CK
{CD}+: CD→E [F2]; E→A [F4]; A→B,C [F1]; B→D [F3]: {A,B,C,D,E} = ALL  → CD is a
CK

Candidate keys: {A}, {E}, {CD}
Prime attributes: A, C, D, E   (appear in some candidate key)
Non-prime attributes: B
```

Check for 3NF violations:

```
FD B→D:  B is not a superkey; D is non-prime  →  3NF VIOLATION (transitive)
All other FDs have superkey or prime-attribute RHS  →  OK for 3NF
```

3NF Decomposition:

```
Remove B→D by separating B and D:
  R1(A, B, C, E)     FDs: A→BC, E→A, (CD→E — CD not fully in R1... keep
relevant FDs)
  Actually, use standard 3NF synthesis:

Canonical cover Fc:
  A → BC  (keep as is; A→B and A→C)
  CD → E
  B → D
  E → A

One relation per FD in canonical cover:
  R1(A, B, C)     F: A→BC             PK: A
  R2(C, D, E)     F: CD→E             PK: CD
  R3(B, D)        F: B→D              PK: B
  R4(A, E)        F: E→A              PK: E

Check: does any relation contain a candidate key of R?
  R1 contains A (CK ✓); R4 contains E (CK ✓)
  → Lossless decomposition guaranteed
```

Final 3NF Schemas:

```
R1(A PK, B, C)          F: A→B, A→C
R2(C, D PK together, E)    F: CD→E
R3(B PK, D)             F: B→D
R4(A, E PK)             F: E→A
```

Q: Normalize R(Stu_id, Stu_name, Dept_id, Dept_name, Crs_id, Crs_name, Grade). FDs: Stu_id→Stu_name; Dept_id→Dept_name; Crs_id→Crs_name,Dept_id; (Stu_id,Crs_id)→Grade

Asked: 2075 Bhadra (Regular Exam) | Marks: [4]

Answer

PK = (Stu_id, Crs_id) [composite key determines all]

Partial FDs (violate 2NF):

```
Stu_id → Stu_name          ← partial (depends on Stu_id only)
Crs_id → Crs_name, Dept_id ← partial (depends on Crs_id only)
Dept_id → Dept_name        ← not a partial FD (Dept_id not in PK)
```

Full FD: (Stu_id, Crs_id) → Grade ✓

2NF Decomposition:

```
R1(Stu_id, Stu_name)              F: Stu_id → Stu_name
R2(Crs_id, Crs_name, Dept_id)    F: Crs_id → Crs_name, Dept_id
R3(Stu_id, Crs_id, Grade)        F: (Stu_id,Crs_id) → Grade
```

Check 3NF — R2 has transitive FD:

```
R2: Crs_id → Dept_id → Dept_name   ← TRANSITIVE
Decompose R2:
  R2a(Crs_id, Crs_name, Dept_id)   F: Crs_id → Crs_name, Dept_id
  R2b(Dept_id, Dept_name)           F: Dept_id → Dept_name
```

Final 3NF Schemas:

```
STUDENT    (Stu_id PK, Stu_name)
COURSE     (Crs_id PK, Crs_name, Dept_id FK→DEPT)
DEPT       (Dept_id PK, Dept_name)
ENROLLMENT (Stu_id FK, Crs_id FK, Grade)   PK=(Stu_id, Crs_id)
```

BCNF DECOMPOSITION WORKED EXAMPLES

Q: What is BCNF? Check if R(A,B,C,D) with FDs AB→C, C→A, C→D is in BCNF. If not, decompose.
Asked: 2082 Kartik (Back Exam) | Marks: [4]

Answer

Find candidate keys:

```
{AB}+: AB→C [F1]; C→A [F2]; C→D [F3]: {A,B,C,D} = ALL  → AB is CK
{CB}+: C→A [F2]; C→D [F3]; AB→C? (A,B ∈ {A,B,C,D}? A yes, B yes): {A,B,C,D} →
CB is CK

Candidate keys: {AB}, {CB}  →  Prime attributes: A, B, C
Non-prime: D
```

Check BCNF for each non-trivial FD:

```
F1: AB → C   Is AB a superkey? {AB}+ = ALL ✓  →  BCNF OK
F2: C  → A   Is C  a superkey? {C}+ = {C,A,D} ≠ ALL ✗  →  BCNF VIOLATION!
F3: C  → D   Is C  a superkey? {C}+ = {C,A,D} ≠ ALL ✗  →  BCNF VIOLATION!
```

BCNF Decomposition using C→A (or C→D):

```
Violating FD: C → A  (use C → AD since C also determines D)

Step 1: Create R1 from the violating FD:
  R1 = {C} ∪ {A, D} = (C, A, D)   FDs: C→A, C→D
  Check R1: C is PK; C→A,D. BCNF ✓

Step 2: R2 = R - {A, D} + {C} = (A,B,C,D) - {A,D} + C = (B, C)
  R2 = (B, C)   FDs: (BC)+ = ? B→? C→A,D but A,D not in R2.
  In R2: only FD is BC→BC (trivial). PK = BC (or B alone if C→something in
R2).
  Actually: AB→C projected to R2(B,C): no non-trivial FD. BC is PK. BCNF ✓

Final decomposition:
  R1(C PK, A, D)     F: C→A, C→D    BCNF ✓
  R2(B, C FK→R1)     F: none non-trivial (BC is PK)   BCNF ✓

Verify lossless: R1 ∩ R2 = {C}; C→A,D in R1 (C is PK of R1) → LOSSLESS ✓
```

Q: Define BCNF. Given R(A,B,C) with FDs A→B, B→C, C→A. Is it in BCNF?
Asked: 2079 Chaitra (Regular Exam) | Marks: [4]

Answer

Find candidate keys:

```
{A}+: A→B; B→C: {A,B,C} = ALL  → A is CK
{B}+: B→C; C→A; A→B: {A,B,C} = ALL  → B is CK
{C}+: C→A; A→B; B→C: {A,B,C} = ALL  → C is CK

ALL three attributes {A}, {B}, {C} are candidate keys!
ALL attributes are prime attributes.
```

Check BCNF:

```
F1: A → B   Is A a superkey? {A}+ = ALL ✓  →  BCNF OK
F2: B → C   Is B a superkey? {B}+ = ALL ✓  →  BCNF OK
F3: C → A   Is C a superkey? {C}+ = ALL ✓  →  BCNF OK

Every determinant is a superkey!  → R(A,B,C) IS IN BCNF ✓

Also: R is in 3NF (all attributes are prime), 2NF, and 1NF.
```

This is an interesting case: a 3-attribute relation where all three attributes cyclically determine each other. Every attribute is a candidate key. No decomposition is needed.

Q: Define BCNF. Given R(A,B,C,D) with FDs A→BCD, B→CD, C→D. Normalize to BCNF.

Asked: 2079 Jestha (Back Exam) | Marks: [4]

Answer

Find candidate keys:

```
{A}+: A→BCD = ALL  →  A is CK (only CK)

Prime attributes: {A}
Non-prime: B, C, D
```

Check BCNF for each non-trivial FD:

```
F1: A → BCD   {A}+ = ALL ✓  →  BCNF OK
F2: B → CD    {B}+ = {B,C,D} ≠ ALL ✗  →  BCNF VIOLATION (B is not superkey)
F3: C → D     {C}+ = {C,D} ≠ ALL ✗  →  BCNF VIOLATION (C is not superkey)
```

BCNF Decomposition (using most restrictive violation first):

```
Step 1: Use violating FD C → D:
  R1 = (C, D)       FDs: C→D  PK=C  BCNF ✓
  R2 = R - {D} + {C} = (A, B, C)   [keep C as link]
```

```
Step 2: Check R2(A,B,C):
  FDs applicable to R2: A→BC (project F); B→C (since B→CD and D not in R2,
B→C)
  {A}+ in R2: A→B,C = ALL of R2  → A is CK of R2
  {B}+ in R2: B→C; no further. {B,C} ≠ R2  → B is not superkey
  FD B→C: B is not superkey of R2 ✗  →  BCNF VIOLATION in R2

Step 3: Use violating FD B → C in R2:
  R2a = (B, C)     FDs: B→C  PK=B  BCNF ✓
  R2b = R2 - {C} + {B} = (A, B)  FDs: A→B  PK=A  BCNF ✓

Final BCNF decomposition:
  R1 (C PK, D)
  R2a(B PK, C FK→R1)
  R2b(A PK, B FK→R2a)
```

This is equivalent to a chain: A→B→C→D, each stored separately.

Q: Normalize R(A,B,C,D) with FDs: AB→C, C→B to BCNF.

Asked: 2077 Chaitra (Regular Exam) | Marks: [4]

Answer

Find candidate keys:

```
Which attributes never appear on RHS? A and D (never determined by others)
→ A and D must be in every CK.

{AD}+: A,D only → {A,D}. Need to find more.
  No FD has A or D on LHS alone.
  AB→C: need B. ADB→C?
{ADB}+: AB→C [have A,B]; C→B [have C]: {A,B,C,D}=ALL  → ADB is superkey.
  Is ADB minimal? Remove D: {AB}+: AB→C; C→B: {A,B,C}≠ALL (missing D)
  Remove B: {AD}+: no applicable FDs. {A,D}≠ALL
  Remove A: {DB}+: no applicable FDs. {D,B}≠ALL
  → ADB is minimal (can't remove any attr). ADB is CK1.

{ADC}+: C→B [have C]: {A,B,C,D}=ALL  → ADC is superkey.
  Remove D: {AC}+: C→B: {A,B,C}≠ALL. Remove A: {DC}+: C→B: {B,C,D}≠ALL.
  → ADC is minimal. ADC is CK2.

Candidate keys: {ABD} and {ACD}
Prime attributes: A, B, C, D — ALL attributes are prime!
```

Check BCNF:

```
F1: AB → C   {AB}+: AB→C; C→B: {A,B,C}≠ALL  →  AB is NOT a superkey ✗  BCNF
VIOLATION
```

```
F2: C  → B   {C}+: C→B; AB→C but A,B needed: {B,C}≠ALL  →  C not superkey ✗
BCNF VIOLATION
```

BCNF Decomposition (use C → B):

```
Violating FD: C → B
  R1 = (C, B)     F: C→B  PK=C  BCNF ✓
  R2 = R - {B} + {C} = (A, C, D)

Check R2(A,C,D): project FDs onto {A,C,D}:
  AB→C: B not in R2, irrelevant
  C→B: B not in R2, irrelevant
  No non-trivial FD applicable to R2 alone.
  → Each attribute determines itself only. PK = ACD. BCNF ✓

Final:
  R1(C PK, B)
  R2(A, C FK→R1, D)   PK = ACD (all three together)

Lossless check: R1 ∩ R2 = {C}; C→B holds in R1 (C is PK of R1) → LOSSLESS ✓
Note: FD AB→C is LOST in decomposition (dependency NOT preserved).
```

Q: Normalize R(A,B,C,D,E) with FDs: AB→CE, E→AB, C→D to BCNF.
Asked: 2074 Bhadra (Regular Exam) | Marks: [6]

Answer

Find candidate keys:

```
{AB}+: AB→CE; C→D: {A,B,C,D,E}=ALL  →  AB is CK
{E}+:  E→AB; AB→CE; C→D: {A,B,C,D,E}=ALL  →  E is CK

Candidate keys: {AB}, {E}
Prime attributes: A, B, E
Non-prime: C, D
```

Check BCNF:

```
AB→CE:  {AB}+=ALL ✓  BCNF OK
E→AB:   {E}+=ALL  ✓  BCNF OK
C→D:    {C}+={C,D}≠ALL  ✗  BCNF VIOLATION (C is not superkey)
```

BCNF Decomposition using C → D:

```
R1 = (C, D)           F: C→D  PK=C  BCNF ✓
R2 = R - {D} + {C} = (A, B, C, E)

Check R2(A,B,C,E) — project FDs:
```

```
  AB→CE: applicable ✓  (A,B,C,E all in R2)
  E→AB:  applicable ✓  (A,B,E all in R2)
  C→D:   D not in R2, not applicable

Candidate keys of R2:
  {AB}+: AB→CE: {A,B,C,E}=ALL of R2 → AB is CK of R2
  {E}+:  E→AB→CE: {A,B,C,E}=ALL of R2 → E is CK of R2

BCNF check on R2:
  AB→CE: AB is superkey of R2 ✓
  E→AB:  E is superkey of R2 ✓
  R2 is in BCNF ✓

Final BCNF decomposition:
  R1(C PK, D)
  R2(A, B, C FK→R1, E)   PKs: AB or E
```

Q: R(A,B,C,D,E) with FDs: AB→C, A→D, D→B, BE→C, B→E. Find all candidate keys. Normalize to BCNF.

Asked: 2073 Bhadra (Regular Exam) | Marks: [4+4]

Answer

Step 1: Find all candidate keys

```
Which attributes never appear on RHS?  A (never on RHS)  →  A in every CK.

{A}+:  A→D; D→B; B→E: {A,D,B,E}; AB→C [have A,B]: {A,B,C,D,E}=ALL
  → {A}+ = ALL  →  A alone is a CANDIDATE KEY!

Candidate keys: {A}  (A alone determines everything)
Prime attributes: {A}
Non-prime: B, C, D, E
```

Step 2: Check BCNF

```
A→D:   {A}+=ALL ✓  BCNF OK
D→B:   {D}+: D→B; B→E: {D,B,E}≠ALL  ✗  BCNF VIOLATION
B→E:   {B}+: B→E: {B,E}≠ALL  ✗  BCNF VIOLATION
AB→C:  {AB}+: AB→C; B→E; A→D; D→B: {A,B,C,D,E}=ALL ✓  BCNF OK
BE→C:  {BE}+: BE→C; B→E already; {B,C,E}≠ALL  ✗  BCNF VIOLATION
```

Step 3: BCNF Decomposition

```
Use D → B (simplest violating FD):
  R1 = (D, B)          F: D→B  PK=D  BCNF ✓
  R2 = R - {B} + {D} = (A, C, D, E)

Check R2(A,C,D,E) — project FDs:
```

```
  A→D ✓ (applicable); B→E? B not in R2;
  AB→C? B not in R2; BE→C? B not in R2
  A→D is only applicable non-trivial FD.
  {A}+ in R2: A→D: {A,D}≠{A,C,D,E}. No more FDs. Need another way to get C,E.

  Hmm — let's reconsider. Project ALL FDs onto R2={A,C,D,E}:
    For each pair X□{A,C,D,E}: compute X+ using F, keep only attrs in
{A,C,D,E}
    A+={A,B,C,D,E} ∩ {A,C,D,E} = {A,C,D,E} → A→CDE in R2
    D+={B,D,E}     ∩ {A,C,D,E} = {D,E}     → D→E in R2
    Others don't give new FDs.

  R2(A,C,D,E): FDs: A→C,D,E;  D→E
  CK of R2: {A}+ in R2 = {A,C,D,E}=ALL → A is CK of R2
  Check BCNF: D→E: {D}+ in R2 = {D,E} ≠ ALL → VIOLATION!

  Use D→E in R2:
  R2a = (D, E)      F: D→E  PK=D  BCNF ✓
  R2b = R2 - {E} + {D} = (A, C, D)
    FDs in R2b: A→C,D; (D→E not applicable, E gone)
    {A}+ in R2b: A→C,D = ALL of R2b → A is CK of R2b. A is superkey for A→C,D
✓ BCNF ✓

Final BCNF decomposition:
  R1 (D PK, B)
  R2a(D PK, E)
  R2b(A PK, C, D FK→R1/R2a)

Or equivalently (merging R1,R2a since both have PK=D):
  Rdept(D PK, B, E)   F: D→B, D→E
  Rmain(A PK, C, D FK→Rdept)
```

Q: Decompose R(A,B,C,D,E,F) into BCNF given FDs: A→BCD, B→DE, C→EF.
Asked: 2071 Bhadra (Regular Exam) | Marks: [4]

Answer

Find candidate keys:

```
Attributes never on RHS: A, C (check: A never RHS; C never RHS)
Wait: C appears on RHS? A→BCD has C on RHS. Yes, A→C.
Attributes never on RHS of ANY FD: check each attr:
  A: on RHS of A→BCD? No. Never on RHS. MUST be in every CK.
  B: on RHS of A→BCD. Can be determined.
  C: on RHS of A→BCD. Can be determined.
  D: on RHS of A→BCD and B→DE.
  E: on RHS of B→DE and C→EF.
  F: on RHS of C→EF.
  → A must be in every CK.

{A}+: A→BCD; B→DE; C→EF: {A,B,C,D,E,F}=ALL  →  A is CK (only CK).
```

```
Prime attributes: {A}. Non-prime: B,C,D,E,F
```

Check BCNF:

```
A→BCD:  A is superkey ✓  BCNF OK
B→DE:   {B}+={B,D,E}≠ALL  ✗  BCNF VIOLATION
C→EF:   {C}+={C,E,F}≠ALL  ✗  BCNF VIOLATION
```

BCNF Decomposition:

```
Use B → DE:
  R1 = (B, D, E)      F: B→DE  PK=B  BCNF ✓
  R2 = R - {D,E} + {B} = (A, B, C, F)

Project FDs onto R2(A,B,C,F):
  A→BC (from A→BCD, project out D): A→B, A→C applicable
  C→F  (from C→EF, project out E): C→F applicable
  {A}+ in R2: A→B,C; C→F: {A,B,C,F}=ALL → A is CK of R2

Check BCNF of R2:
  A→BC: A is superkey ✓
  C→F:  {C}+ in R2 = {C,F} ≠ ALL ✗  VIOLATION

Use C → F in R2:
  R2a = (C, F)        F: C→F  PK=C  BCNF ✓
  R2b = R2-{F}+{C} = (A,B,C)  FDs: A→B,A→C
     {A}+ in R2b = {A,B,C}=ALL → A is CK ✓  A→B,A→C: A is superkey ✓  BCNF ✓

Final BCNF decomposition:
  R1 (B PK, D, E)
  R2a(C PK, F)
  R2b(A PK, B FK→R1, C FK→R2a)
```

Q: Decompose R(A,B,C,D,E,F) into BCNF given: A→BC, B→CD, C→DE, CE→F.
Asked: 2075 Baisakh (Back Exam) | Marks: [4]

Answer

Find candidate key:

```
A never on RHS → A in every CK.
{A}+: A→BC; B→CD; C→DE; CE→F [C,E in closure now]: {A,B,C,D,E,F}=ALL → A is
CK.

CK = {A}. All non-A attributes are non-prime.
```

BCNF violations:

```
A→BC:   A is superkey ✓
B→CD:   {B}+ = {B,C,D,E,F} (B→C→D,E; CE→F) ≠ ALL (missing A) ✗
C→DE:   {C}+ = {C,D,E,F} ≠ ALL ✗
CE→F:   {CE}+ = {C,D,E,F} ≠ ALL ✗
```

Decomposition (pick simplest violating FD first: C→DE):

```
Use C → DE:
  R1 = (C, D, E)     check: CE→F applicable? E in R1: {CE}+=? CE→F→{C,D,E,F} ≠
ALL of R1
  Actually project onto R1={C,D,E}: CE→F: F not in R1. Only C→DE applies.
  PK of R1: C (C→DE covers all). BCNF ✓

  R2 = R - {D,E} + {C} = (A,B,C,F)
  Project FDs onto {A,B,C,F}: A→BC ✓; B→C (from B→CD, project out D); CE→F→C→?
(E not in R2)
  Hmm: {CE}+ projected: CE not in R2. CE→F not applicable (E gone).
  FDs in R2: A→B,C; B→C
  {A}+ in R2: A→B,C: {A,B,C} but F missing. No FD gives F. → F is in R2 but
not determined!
  → A alone doesn't determine F in R2. PK of R2 must include F? Or is F
determined somehow?
  Original CE→F: C in R2, E not in R2. CE→F lost. F becomes a standalone
attribute.
  → PK of R2: (A,F) [since nothing determines F]? Or just check if A→F
possible.
  Not determined → F must be added to PK: PK of R2 = AF? No, check properly.

  Let's use a cleaner approach — use B→CD next on R2:
  B→C: {B}+ in R2={A,B,C,F}: B→C; then nothing more: {B,C}≠ALL. B not superkey
in R2 ✗

Use B→C in R2:
  R2a = (B, C FK→R1)   F: B→C  PK=B  BCNF ✓
  R2b = R2-{C}+{B} = (A,B,F)  F: A→B (from A→BC, project out C)
    {A}+ in R2b: A→B: {A,B}≠{A,B,F}. F still undetermined.
    PK of R2b: (A,F) since nothing determines F. A→B: A is not superkey. ✗
    Use A→B in R2b:
    R2b1=(A,B) F:A→B PK=A BCNF ✓
    R2b2=(A,F) F: none (trivial). PK=(A,F) BCNF ✓

Final BCNF decomposition:
  R1  (C PK, D, E)
  R2a (B PK, C FK)
  R2b1(A PK, B FK)
  R2b2(A, F)  PK=(A,F)
```

Note: CE→F cannot be preserved in this decomposition since C and E end up in different relations. This illustrates that BCNF decomposition may not always preserve all dependencies.

LOSSLESS DECOMPOSITION & DEPENDENCY PRESERVATION

Q: What is lossless join decomposition? When is a decomposition lossless? Explain the test with examples.
Asked: 2082 Kartik; 2079 Chaitra; 2077; 2074; 2072 Ashwin; 2071; 2070 (all years) | Marks: [4]

Answer

Lossless Join Decomposition

A decomposition of relation R into schemas R1 and R2 is lossless (or lossless-join) if for every valid instance r of R:

```
r = πR1(r) □ πR2(r)

That is: joining the projections of r onto R1 and R2 gives back the original
relation r
without any spurious (false) tuples.
```

A decomposition is lossy if the natural join of the projected relations produces extra spurious tuples that were not in the original relation — information is distorted (not lost, but fabricated).

Test for Lossless Decomposition (2-Relation Case)

For a decomposition of R into R1 and R2, the decomposition is lossless if and only if at least one of the following holds:

```
Condition 1: (R1 ∩ R2) → R1     (common attributes determine all of R1)
Condition 2: (R1 ∩ R2) → R2     (common attributes determine all of R2)

i.e., the set of common attributes must be a superkey of R1 OR R2.
```

Example 1 — Lossless Decomposition:

```
R(A, B, C, D)  with FD: A → BC
Decomposition: R1(A, B, C)  and  R2(A, D)

R1 ∩ R2 = {A}
Is {A} → R1(A,B,C)? i.e., A → A,B,C?
  A → BC (given), so A → A,B,C ✓
→ Condition 1 satisfied → LOSSLESS ✓

Sample instance of R:
  A   B   C   D
  a1  b1  c1  d1
  a1  b1  c1  d2
  a2  b2  c2  d3

πR1: (a1,b1,c1),(a2,b2,c2)  πR2: (a1,d1),(a1,d2),(a2,d3)
Natural join on A: (a1,b1,c1,d1),(a1,b1,c1,d2),(a2,b2,c2,d3) = original R ✓
```

Example 2 — Lossy Decomposition:

```
R(A, B, C)   with no FDs between {B} and {A,C}
Decomposition: R1(A, B)  and  R2(B, C)

R1 ∩ R2 = {B}
Is B → R1(A,B)? i.e., B → A? NOT given → NO
Is B → R2(B,C)? i.e., B → C? NOT given → NO
→ LOSSY DECOMPOSITION ✗

Sample instance:
  A   B   C
  a1  b1  c1
  a2  b1  c2

πR1: (a1,b1),(a2,b1)   πR2: (b1,c1),(b1,c2)
Natural join: (a1,b1,c1),(a1,b1,c2),(a2,b1,c1),(a2,b1,c2)
→ 4 tuples but original had only 2! Spurious tuples generated. LOSSY ✗
```

General Lossless Test (Chase Algorithm — Multiple Relations)

For a decomposition into more than 2 relations, the tabular (chase) method is used:

```
Algorithm:
1. Create a table with one row per Ri (decomposed relation)
   and one column per attribute of R.
2. Fill in each cell:
   - If attribute Aj is in Ri: enter 'aj' (distinguished symbol)
   - If not: enter 'bij' (non-distinguished symbol, unique per row i, column j)
3. Repeatedly apply FDs: for each FD X→Y and two rows that agree on X,
   make them agree on Y too (prefer distinguished symbols 'a').
4. If after all applicable FDs, any row becomes all-distinguished (a1,a2,...,an),
   the decomposition is LOSSLESS.
```

Worked Example — Chase Test:

```
R(A,B,C,D), FDs: A→B, B→C
Decompose into: R1(A,B), R2(B,C), R3(A,D)

Initial table:
     A     B     C     D
R1:  a1    a2    b13   b14
R2:  b21   a2    a3    b24
R3:  a1    b32   b33   a4

Apply A→B: R1 and R3 both have A=a1:
  Make B the same: R3.B = a2 (prefer distinguished)
     A     B     C     D
R1:  a1    a2    b13   b14
R2:  b21   a2    a3    b24
R3:  a1    a2    b33   a4    ← B updated to a2

Apply B→C: R1, R2, R3 all have B=a2:
  Make C the same: R1.C = a3, R3.C = a3 (prefer a3 from R2)
     A     B     C     D
R1:  a1    a2    a3    b14
R2:  b21   a2    a3    b24
R3:  a1    a2    a3    a4    ← Row R3 is now a1,a2,a3,a4 = ALL DISTINGUISHED

Row R3 = (a1, a2, a3, a4) → LOSSLESS DECOMPOSITION ✓
```

Q: Explain dependency preservation in database decomposition. Why is it important?
Asked: 2072 Ashwin (Regular Exam) | Marks: [4]

Answer

Dependency Preservation

A decomposition of R into R1, R2, ..., Rn is dependency-preserving if the set of FDs F implied by the individual decomposed schemas (F1 ∪ F2 ∪ ... ∪ Fn) is equivalent to the original set F. That is, no FD from F is lost in the decomposition.

```
Formally: (F1 ∪ F2 ∪ ... ∪ Fn)+ = F+

where Fi = projection of F onto Ri = {X→Y ∈ F+ | X,Y ⊆ Ri}
```

Why Dependency Preservation Matters

- Enforcement efficiency: If a dependency is preserved, it can be checked by examining a single relation — without performing a JOIN.
- Data integrity: If a dependency is lost, the DBMS cannot enforce it within a single table; it would require joining multiple tables and adding triggers/constraints.
- Update anomalies: Without dependency preservation, data inconsistencies can slip in undetected through insertions/updates to the decomposed relations.

Classic Example — Dependency Not Preserved:

```
Teaches(student, course, instructor)
  FDs: (student,course) → instructor; instructor → course

BCNF Decomposition (on instructor → course):
  R1(instructor, course)   — preserves instructor→course
  R2(student, instructor)  — preserves (student,instructor)→?

Lost FD: (student, course) → instructor
  This FD spans both R1 and R2 — cannot be checked without a JOIN.
  To verify it, we'd need: JOIN R1 and R2 on instructor, then check.

Consequence: A student could be enrolled with different instructors for the
  same course — inconsistency that the DBMS cannot prevent automatically.
```

3NF vs BCNF: Dependency Trade-off

Property	3NF Decomposition	BCNF Decomposition
Lossless join	Always achievable	Always achievable
Dependency preservation	Always achievable	May NOT be achievable
Eliminates all anomalies	No — some redundancy may remain (prime attr violations)	Yes — all FD-based anomalies removed
Preferred when...	Dependency preservation is critical	Strict redundancy elimination is required

MULTI-VALUED DEPENDENCIES AND 4NF

Q: Define multi-valued dependency (MVD). Explain Fourth Normal Form (4NF) with examples.
Asked: 2079 Jestha; 2075 Bhadra; 2073 Magh; 2070 Magh | Marks: [4]

Answer

Multi-Valued Dependency (MVD)

A multi-valued dependency (MVD) X →→ Y (read 'X multi-determines Y') holds in relation R if, for any two tuples t1 and t2 in R where t1[X] = t2[X], there also exist tuples t3 and t4 such that:

```
t3[X] = t4[X] = t1[X] = t2[X]
t3[Y] = t1[Y]   and   t3[Z] = t2[Z]
t4[Y] = t2[Y]   and   t4[Z] = t1[Z]

where Z = R - X - Y (the remaining attributes)

In simpler terms: X →→ Y means that for a given value of X,
the set of Y-values is determined independently of the Z-values.
```

Intuitive Explanation:

An MVD arises when a single entity is associated with multiple independent sets of values for two different attributes. The two sets are independent — knowing one doesn't affect the other.

Classic Example:

```
CourseTeacher(course, teacher, textbook)

Suppose: a course can have multiple teachers AND multiple textbooks,
         independently of each other.

Data:
  course     teacher    textbook
  CS101      Alice      Database Systems
  CS101      Alice      Operating Systems
  CS101      Bob        Database Systems
  CS101      Bob        Operating Systems

MVDs: course →→ teacher    (for CS101, teachers are {Alice, Bob} — independent of
textbook)
      course →→ textbook   (for CS101, textbooks are {DS, OS} — independent of
teacher)

If Alice teaches CS101, she uses ALL CS101 textbooks (and vice versa).
This causes redundancy — every (course,teacher) pair repeats for every textbook.
```

Properties of MVDs

- Every FD is also an MVD: If X → Y, then X →→ Y (trivially, since each X-value has exactly one Y-value)
- Complementation: If X →→ Y in R, then X →→ (R − X − Y) also holds in R
- Trivial MVD: X →→ Y is trivial if Y ⊆ X or X ∪ Y = R

- MVDs arise when a relation has more than one independent multi-valued attribute

Fourth Normal Form (4NF)

A relation R is in 4NF if it is in BCNF AND for every non-trivial MVD X →→ Y in R, X is a superkey of R.

```
4NF condition: For every non-trivial MVD X →→ Y:
               X must be a superkey of R

This eliminates redundancy caused by independent multi-valued attributes.
```

Example — CourseTeacher NOT in 4NF:

```
CourseTeacher(course, teacher, textbook)
  PK = (course, teacher, textbook)  [no proper subset is a key]

MVD: course →→ teacher   (course is NOT a superkey — course alone doesn't identify
full row)
MVD: course →→ textbook  (same violation)

→ NOT in 4NF
```

4NF Decomposition:

```
Decompose on course →→ teacher:
  R1(course, teacher)    — stores which teachers teach each course
  R2(course, textbook)   — stores which textbooks each course uses

Now: no MVD in R1 (course→teacher is an FD here); no MVD in R2.
→ Both R1 and R2 are in 4NF ✓

Original 4 rows become:
R1: (CS101,Alice), (CS101,Bob)
R2: (CS101,Database Systems), (CS101,Operating Systems)

To reconstruct: R1 □ R2  generates all 4 original combinations — LOSSLESS ✓
```

Normal Form	Violation Removed	Condition
1NF	Non-atomic attributes	All domains are atomic
2NF	Partial FDs	Every non-key attr fully depends on PK
3NF	Transitive FDs	Every determinant of a non-key attr is a superkey, OR Y is prime
BCNF	All FD anomalies	Every determinant is a superkey (no exceptions)
4NF	MVD redundancy	For every non-trivial MVD X→→Y, X is a superkey
5NF (PJNF)	Join dependency redundancy	Every join dependency is implied by candidate keys

SUMMARY TABLE — Chapter 4 Question Coverage

All Chapter 4 questions from TU-IOE BCT examination papers (2070–2082) and their coverage in this document.

No.	Question / Topic	Asked In	Marks
C1	Functional Dependency: definition, trivial/non-trivial, full/partial/transitive	2078 Poush, 2077 Chaitra, 2075 Baisakh, 2074, 2072 Magh, 2070–2081 (all)	[4]
C2	Armstrong's Axioms (reflexivity, augmentation, transitivity) + derived rules	2081 Chaitra, 2080, 2079 Ashwin, 2076, 2075, 2073 Magh, 2071, 2070–2081	[4]
C3	Closure of attribute set; closure of FD set; canonical cover	2081 Ashwin, 2075, 2074, 2073 Magh, 2072 Ashwin, 2071 Magh	[4+4]
C4	Anomalies (insertion, deletion, update) and why normalization is needed	2078 Baishakh, 2073 Bhadra, 2070 Bhadra, 2078 Chaitra	[4]
C5	1NF, 2NF, 3NF — definitions, examples, step-by-step normalization	2081 Chaitra, 2080, 2079 Chaitra, 2078 Poush, 2078 Baishakh, 2073, 2070 (all)	[6]
C6	BCNF — definition, BCNF decomposition algorithm, 3NF vs BCNF	2079 Chaitra, 2078 Poush, 2078 Baishakh, 2077, 2076, 2072 Magh, 2071	[4]
N1	Normalize to 3NF: Student-Course-Instructor (2081 Ashwin)	2081 Ashwin	[6]
N2	Normalize to 3NF: Parts-Suppliers (P#,S#,Qty	2080 Ashwin, 2073 Magh	[6]
N3	Normalize to 3NF: Employee-Project-Department (ENo/EId/PNo/DNo)	2079 Ashwin, 2078 Chaitra, 2075 Baisakh	[6-8]
N4	Normalize to 3NF: R(A,B,C,D,E): A→BC, B→D, A→E	2082 Kartik	[6]
N5	Normalize to 3NF: R(A,B,C,D,E): A→BC, CD→E, B→D, E	2076 Baisakh	[4]
N6	Normalize to 3NF: Student(Stu_id,Dept_id,Crs_id,Grade)	2075 Bhadra	[4]
B1	Normalize to BCNF: R(A,B,C,D): AB→C, C→A, C→D	2082 Kartik	[4]
B2	Normalize to BCNF: R(A,B,C): A→B, B→C, C→A	2079 Chaitra	[4]
B3	Normalize to BCNF: R(A,B,C,D): A→BCD, B→CD, C→D	2079 Jestha	[4]
B4	Normalize to BCNF: R(A,B,C,D): AB→C, C→B	2077 Chaitra	[4]
B5	Normalize to BCNF: R(A,B,C,D,E): AB→C, C→A, B→D, E→A	2076 Baisakh	[4]
B6	Normalize to BCNF: R(A,B,C,D,E): AB→CE, E→AB, C→D	2074 Bhadra	[6]
B7	Normalize to BCNF: R(A,B,C,D,E): AB→C, A→D, D→B, BE→C, B→E — 2073	2073 Bhadra	[4+4]
B8	Normalize to BCNF: R(A,B,C,D,E,F): A→BCD, B→DE, C→EF	2071 Bhadra	[4]
B9	Normalize to BCNF: R(A,B,C,D,E,F): A→BC, B→CD, C→DE, CE→F	2075 Baisakh	[4]
L1	Lossless join decomposition — test using FD-based condition (2-relation case)	2082, 2079 Chaitra, 2077, 2074, 2072 Ashwin, 2071, 2070	[4]
L2	Dependency preservation — definition and significance	2072 Ashwin	[4]
M1	Multi-valued dependency and 4NF	2079 Jestha, 2075 Bhadra, 2073 Magh,	[4]

Notation used throughout: FD = Functional Dependency; PK = Primary Key; CK = Candidate Key; MVD = Multi-Valued Dependency. The closure algorithm always starts with X+ = X (reflexivity). A single-attribute PK means the relation is automatically in 2NF. Armstrong's Axioms (Reflexivity, Augmentation, Transitivity) are sound and complete.

— End of Chapter 4 Answer Bank —